

Plugin	MBR Login Customiser
Audited version	1.1.0
Remediated version	1.1.1
Audit date	June 2026
Performed by	Made by Robert (internal review)
Scope	Full source review of mbr-custom-login.php, admin.js and packaged readme files

Summary

MBR Login Customiser was given a full manual source-code security review. The plugin was found to be well structured: it consistently uses WordPress capability checks, settings-API nonces, output escaping and per-setting sanitisation callbacks. No Critical or High severity issues were found in the default configuration, and no unauthenticated code execution, SQL injection or stored cross-site scripting was identified.

Six issues were identified and fixed in version 1.1.1: two of medium severity, three of low severity and one hardening change. The most significant finding concerns the optional “behind a proxy/CDN” mode, where the brute-force lockout could previously be evaded or weaponised using spoofed HTTP headers. Two further items are recorded as informational notes rather than defects.

All identified issues have been remediated.

Ref	Severity	Issue	Status
MBR-01	Medium	Brute-force lockout bypass via spoofable proxy headers	Fixed in 1.1.1
MBR-02	Medium	Unbounded growth of the failed-attempt store	Fixed in 1.1.1
MBR-03	Low	Non-constant-time comparison of the emergency key	Fixed in 1.1.1
MBR-04	Low	CSS-injection vector via gradient direction value	Fixed in 1.1.1
MBR-05	Low	Login pass-through cookie missing SameSite attribute	Fixed in 1.1.1
MBR-06	Low	Request data used without wp_unslash()	Fixed in 1.1.1
MBR-07	Info	Emergency key sent as a query parameter	By design — note added
MBR-08	Info	admin-post.php blocked for logged-out users	Functional note

Scope and methodology

The review was a manual, white-box read of the complete plugin source. The following classes of issue were checked for:

- Authentication and authorisation bypass, and capability checks on every privileged action.
- Cross-site request forgery (CSRF) on state-changing requests.

- Cross-site scripting (XSS), both reflected and stored, on every value that reaches HTML, attribute, URL or CSS output contexts.
- SQL injection and unsafe database access.
- Insecure direct object references and missing access controls.
- Server-side request forgery and unsafe file inclusion.
- Secret handling, comparison, transport and storage.
- Input trust boundaries, including client-controllable HTTP headers.
- Denial-of-service and resource-exhaustion paths.

WordPress core handles the persistence layer through the Settings and Options APIs, so there is no hand-written SQL in the plugin; SQL injection is therefore not applicable. All administrative screens are gated behind the `manage_options` capability, and settings forms are protected by the core Settings API nonce. The single custom state-changing action (clearing a logout) was confirmed to verify a nonce and run only inside a capability-checked screen.

Findings

MBR-01 — Brute-force logout bypass via spoofable proxy headers **[Medium]**

Affected: login attempt limiting, when the “behind a proxy/CDN” option is enabled.

The login attempt limiter identifies a visitor by IP address. With the proxy/CDN option enabled, the previous code walked a list of HTTP headers (CF-Connecting-IP, True-Client-IP, X-Forwarded-For, X-Real-IP) and trusted the first valid value it found.

Any of those headers can be set freely by a client that reaches the origin server directly. An attacker could therefore send a different forged value on each request to evade the logout entirely, or send a chosen victim’s address to have that address locked out (a targeted denial of service against, for example, an administrator). The logout is the plugin’s core protective control, so a bypass of it is the most material finding in this review.

In the default configuration the option is off and the server-provided `REMOTE_ADDR` is used, which a client cannot forge. The risk is therefore limited to sites that enable the proxy option. On a site whose origin IP is reachable directly — a common situation that undermines CDN-only access — the practical impact rises towards High.

Remediation. The plugin now reads the client IP from a single, administrator-selected trusted header rather than guessing across several. A new “Trusted IP Header” setting on the Security tab lets the administrator pick the header their proxy actually sets (Cloudflare, Akamai/True-Client-IP, nginx X-Real-IP, or generic X-Forwarded-For), and the value is validated as a real IP. The documentation now states that the origin must only accept traffic from the chosen proxy for the header to be trustworthy.

MBR-02 — Unbounded growth of the failed-attempt store **[Medium]**

Affected: login attempt limiting storage.

Failed-attempt and logout records are kept in a single WordPress option that is read and written on each failed login. Expired records are pruned on read, but within an active window there was no ceiling on the number of distinct IP records. A distributed brute-force attack from many source addresses could grow that single row to a large size, which is then loaded and rewritten on every attempt — a self-amplifying load and database-bloat condition.

Remediation. The store is now capped at a fixed number of tracked addresses. Active lockouts are never evicted; once the cap is reached, the least recently seen non-locked records are dropped first. This bounds the row size while preserving every live lockout.

MBR-03 — Non-constant-time comparison of the emergency key [Low]

Affected: emergency access URL.

The emergency access key was compared to the user-supplied value with a standard string comparison, which can return early on the first differing byte. In theory this leaks timing information about a secret. In practice the key is a 32-character random string and recovering it by network timing is not feasible, and even if known it only reveals the hidden login location rather than granting any authenticated access. The severity is therefore low, but constant-time comparison of secrets is a clear best practice.

Remediation. The comparison now uses `hash_equals()`, and the supplied value is unslashed and cast to a string before comparison.

MBR-04 — CSS-injection vector via gradient direction value [Low]

Affected: login page appearance (authenticated administrator).

The gradient direction was saved with a generic text sanitiser and then written into a CSS `linear-gradient(...)` context using attribute escaping. Attribute escaping does not neutralise CSS metacharacters such as parentheses and braces, so a value posted directly to the options endpoint could break out into arbitrary CSS. The setting is administrator-only, and an administrator already has a dedicated Custom CSS field, so this does not cross a privilege boundary; it is recorded for completeness and clean output handling.

Remediation. The gradient direction is now constrained to a fixed allowlist both at save time and at output time, so only known-good values can ever reach the stylesheet.

MBR-05 — Login pass-through cookie missing SameSite attribute [Low]

Affected: the short-lived cookie that admits a visitor to the real login page.

The pass-through cookie was set with the Secure and HttpOnly flags but without a SameSite attribute.

Remediation. The cookie is now issued with `SameSite=Lax` on PHP 7.3 and above, with a fallback to the legacy cookie signature on older PHP so the plugin continues to function on its stated minimum of PHP 7.0.

MBR-06 — Request data used without `wp_unslash()` [Low]

Affected: request parsing.

A small number of reads from `$_SERVER` and `$_GET` were used without `wp_unslash()`, which is the WordPress convention for handling the slashes that core adds to superglobals. This is a correctness and hardening matter rather than a directly exploitable flaw.

Remediation. `wp_unslash()` is now applied to the request URI, query string, emergency parameter and proxy header values before use.

MBR-07 — Emergency key sent as a query parameter [Info]

The emergency access URL passes the key as a query-string parameter, which can appear in server access logs, browser history and referrer headers. This is inherent to the “paste a URL to recover access” design and is consistent with how comparable recovery links work. No change was made beyond constant-time comparison (MBR-03). Administrators who rotate the key after use, or who avoid sharing the URL over channels that retain it, reduce this exposure.

MBR-08 — admin-post.php blocked for logged-out users [Info]

When standard-login blocking is active, requests to wp-admin for logged-out visitors return a 404. This also covers admin-post.php, which some themes and plugins use for front-end form handlers aimed at unauthenticated users. This is a functional consideration rather than a security defect and is noted so that site owners using such handlers are aware of it.

Conclusion

MBR Login Customiser follows WordPress security conventions throughout and contained no Critical or High severity vulnerabilities in its default configuration. The issues identified were concentrated in the optional proxy-aware brute-force limiter and in a set of low-severity hardening items, all of which have been addressed in version 1.1.1. Users are encouraged to update, and — if they run behind Cloudflare or another CDN — to review the new Trusted IP Header setting on the Security tab.

Security review is an ongoing process. Suspected issues can be reported via littlewebshack.com.